Systemy operacyjne

Lista zadań nr 1

Na zajęcia 13-15 października 2026

Należy przygotować się do zajęć czytając następujące rozdziały książek:

- Arpaci-Dusseau: Processes[1], Process API[2], Address Spaces[3]
- Tanenbaum (wydanie czwarte): 2.1, 10.3, 11.4

UWAGA! W trakcie prezentacji należy być gotowym do zdefiniowania pojęć oznaczonych **wytłuszczoną** czcionką.

Podręcznikiem do zadań praktycznych jest „*Advanced Programming in the UNIX Environment*" (w skrócie APUE) oraz „*The Linux Programming Interface*" (w skrócie LPI). Proszę przede wszystkim korzystać z rozdziałów 2 i 3 anglojęzycznego podręcznika systemowego – np. `man 2 read`. Kiedy jego treść nie jest wystarczająco klarowna, to warto spojrzeć na rozdziały opisujące analogiczne tematy w podręczniku FreeBSD – np. `man 2freebsd read`.

Wskazówka: W systemie bazującym na Debianie zainstaluj pakiety `manpages-dev`, `manpages-posix-dev`, `freebsd-manpages`.

Rozwiązania zadań należy prezentować przy pomocy rzutnika lub telewizora (w zależności od sali). Każde rozwiązanie należy starannie przygotować do prezentacji przed zajęciami – najlepiej w postaci pliku tekstowego z listą poleceń do wykonania i komentarzami. W przypadku zbędnego przeciągania czasu odpowiedzi ze względu na problemy techniczne prowadzący ma prawo **skreślić zadanie i postawić jeden punkt ujemny**.

UWAGA! Każdy student **musi** przygotować skrypt (lub inne gotowe polecenie), który przy pomocy programu `xrandr`[4] (na X11) – lub programu specyficznego dla używanego kompozytora zgodnego z protokołem Wayland – ustawi rozdzielczość ekranu wbudowanego na 1920×1080 i sklonuje go na zewnętrzne złącze `HDMI` (lub `VGA` jeśli grupa korzysta z rzutnika, a nie telewizora).

UWAGA 2! Każdy student **musi** przygotować skrypt (lub inne gotowe polecenie bądź konfigurację), który szybko i sprawnie dla używanego programu terminala należy ustawi dużą czcionkę (około 32 wierszy w trybie pełnoekranowym), kontrastowe kolory i jasne tło.

UWAGA 3! Studenci z laptopami mającymi złącza inne niż `HDMI` (lub `VGA` w grupach z rzutnikiem) muszą mieć podczas ćwiczeń działającą przejściówkę do jednego z wymienionych złącz, w przypadku braku, prowadzący może skreślić zadanie i postawić jeden punkt ujemny.[5] [6]

[1] `http://pages.cs.wisc.edu/~remzi/OSTEP/cpu-intro.pdf`
[2] `http://pages.cs.wisc.edu/~remzi/OSTEP/cpu-api.pdf`
[3] `http://pages.cs.wisc.edu/~remzi/OSTEP/vm-intro.pdf`
[4] `https://wiki.archlinux.org/index.php/xrandr`
[5] Warto zaznaczyć, że postawienie ujemnych punktów, nie wyklucza wybrania studenta do innego zadania, co w przypadku dalszej niemożności zaprezentowania rozwiązania, może skutkować – w zależności od decyzji prowadzącego – dalszymi skreśleniami i dalszymi punktami ujemnymi.
[6] Jeśli ktoś ma (pomocnych) znajomych, to zadania można prezentować na cudzym komputerze. Tylko że część zadań zapewne będzie wymagała «sudo»...

Zadanie 1. W systemach uniksowych wszystkie procesy są związane relacją **rodzic-dziecko**. Uruchom polecenie «`ps -eo user,pid,ppid,pgid,tid,pri,stat,wchan,cmd`». Na wydruku zidentyfikuj **identyfikator procesu**, **identyfikator grupy procesów**, **identyfikator rodzica** oraz **właściciela** procesu. Kto jest rodzicem procesu `init`? Wskaż, które z wyświetlonych zadań są **wątkami jądra**. Jakie jest znaczenie poszczególnych znaków w kolumnie `STAT`? Wyświetl drzewiastą reprezentację **hierarchii procesów** poleceniem `pstree` – które z zadań są wątkami?

Wskazówka: Może się przydać specyfikacja POSIX `https://pubs.opengroup.org/onlinepubs/9799919799/` .

Zadanie 2. Jak jądro systemu reaguje na sytuację kiedy proces staje się **sierotą**? W jaki sposób pogrzebać proces, który wszedł w stan **zombie**? Czemu proces nie może sam siebie pogrzebać? Zauważ, że proces może, przy pomocy `waitpid(2)`, czekać na zmianę **stanu** wyłącznie swoich dzieci. Co złego mogłoby się stać, gdyby znieść to ograniczenie? Rozważ scenariusze (a) dziecko może czekać na zmianę stanu swojego rodzica (b) wiele procesów oczekuje na zmianę stanu jednego procesu.

Wskazówka: Proces wykonujący w jądrze implementację wywołania systemowego `_exit(2)` nie może zwolnić stosu jądra, na którym się wykonuje. Kto zatem musi to zrobić?

Zadanie 3. Do czego służy system plików `proc(5)` w systemie Linux? Dla wybranego przez siebie procesu o identyfikatorze pid wydrukuj zawartość katalogu «`/proc/pid`». Wyświetl plik zawierający **argumenty programu** oraz **zmienne środowiskowe**. Podaj znaczenie następujących pól pliku «`status`»: `Uid`, `Gid`, `Groups`, `VmPeak`, `VmSize`, `VmRSS`, `Threads`, `voluntary_ctxt_switches`, `nonvoluntary_ctxt_switches`. Czym się różni PSS od RSS (pokazane są one w pliku «`smaps`»)?

UWAGA! Prowadzący ćwiczenia nie zadowoli się cytowaniem podręcznika systemowego – trzeba wykazać się dociekliwością!

Zadanie 4. Wyjaśnij, czym są i czym się różnią: menadżer okien (ang. *window manager*), menadżer wyświetlania (ang. *display manager*), środowisko pulpitu (ang. *desktop environment*) i serwer wyświetlania (ang. *display server*).

Znajdź pid procesu X-serwera[7] (jeśli używasz X11) lub kompozytora[8] (jeśli korzystasz z Waylanda), a następnie używając polecenia «`pmap`» wyświetl zawartość jego przestrzeni adresowej. Zidentyfikuj w niej poszczególne *zasoby pamięciowe* – tj. stos, stertę, **segmenty programu**, **pamięć anonimową**, **pliki odwzorowane w pamięć**. Należy wyjaśnić znaczenie kolumn wydruku!

Zadanie 5. Używając programu «`lsof`» wyświetl **zasoby plikopodobne** podpięte do procesu przeglądarki «`firefox`». Wyjaśnij znaczenie poszczególnych kolumn wykazu, po czym zidentyfikuj **pliki zwykłe**, **katalogi**, **urządzenia**, **gniazda** (sieciowe lub domeny uniksowej), **potoki** i **pliki anonimowe** (ang. *anonymous inode*). Przekieruj wyjście z programu «`lsof`», przed i po otwarciu wybranej strony, odpowiednio do plików «`before`» i «`after`». Czy poleceniem «`diff -u before after`» jesteś w stanie zidentyfikować nowo utworzone połączenia sieciowe? W jaki sposób uzyskać listę zasobów plikopodbnych podpiętych do procesu bez «`lsof`»?

Wskazówka: Może coś ciekawego jest w `procfs`?

Zadanie 6. Wbudowanym poleceniem powłoki «`time`» zmierz **czas wykonania** długo działającego procesu, np. polecenia «`find /usr`». Czemu suma czasów user i sys (a) nie jest równa real (b) może być większa od real? Poleceniem «`ulimit`» nałóż **ograniczenie** na **czas wykonania** procesów potomnych powłoki tak, by limit się wyczerpał. Uruchom ponownie wybrany program – który sygnał wysłano do procesu?

Zadanie 7. (0,5 p.) Odpowiedz na poniższe pytania:

- Jaki jest minimalny **kwant czasu** procesora w Twoim systemie operacyjnym?
- W jaki sposób dzielone jest wykorzystanie karty graficznej? Z użyciem czasu (ang. ***time sharing***) czy z użyciem przestrzeni (ang. ***space sharing***)?
- Uruchom «`top`» i posortuj malejąco po wartościach kolumny `VIRT`. Policz sumę, dla pierwszych 5 procesów, porównaj z dostępną pamięcią RAM na Twoim komputerze, a następnie skomentuj otrzymane rezultaty.

[7] `https://en.wikipedia.org/wiki/X_Window_System`
[8] `https://wiki.archlinux.org/title/Wayland#Compositors`

Zadanie 8. (bonusowe) Opisz w jaki sposób działa domyślny planista (ang. *scheduler*) w jądrze Linuksa, czyli EEVDF. Jakie dane zawiera «`/sys/kernel/debug/sched/debug`»?

Ściągnij ze strony przedmiotu pobierz archiwum «`so26_lista_1.tar.gz`», następnie rozpakuj i zapoznaj się z dostarczonymi plikami.

Zadanie 9. Napisz program, który będzie prezentował, że pliki procesu są **kopiowane przez referencję** w trakcie wywołania `fork(2)`. W procesie głównym otwórz plik do odczytu `open(2)`. Czy zamknięcie pliku `close(2)` w procesie głównym zamyka plik także w dziecku? Czy odczyt z pliku `read(2)` zmienia **pozycję kursora** `lseek(2)` w drugim procesie? Wyjaśnij zachowanie swojego programu!

Przed każdym komunikatem diagnostycznym wypisz pid procesu. W drugiej części zadania należy wydrukować bieżącą pozycję kursora pliku przed operacją odczytu z pliku. Należy wykorzystać dostarczone funkcje opakowujące uniksowe wywołania systemowe z biblioteki `libcsapp`.

Wskazówka: Zagadnienie opisano w APUE rozdział 8.3.

Zadanie 10. (Pomysłodawcą zadania jest Piotr Polesiuk.)

Rozwiąż problem n hetmanów[9] z użyciem `fork(2)` i `waitpid(2)`. Gdy w i-tym elemencie tablicy «`board`» przechowywana jest wartość j znaczy to, że pozycja i-tego hetmana na szachownicy to (i, j). Mając niekonfliktujące ustawienie pierwszych $k-1$ hetmanów po kolei startuj n podprocesów z proponowanym ustawieniem k-tego hetmana. Podproces, który wykryje konfliktujące ustawienie hetmanów, ma zakończyć swe działanie. W przeciwnym wypadku zachowuje się jak rodzic dla $k+1$ hetmana. Podproces, który uzyska prawidłowe ustawienie n hetmanów, ma wydrukować je na standardowe wyjście. Procedura «`ndselect`» wraca wielokrotnie z kolejnymi liczbami z zakresu $0 \ldots n-1$.

Linie wydruków plansz z prawidłowymi ustawieniami hetmanów nie mogą się przeplatać. Uważaj, żeby przez przypadek nie zaprogramować **fork bomby**[10]!

UWAGA! Należy wytłumaczyć działanie programu rysując diagram procesów pokazany na wykładzie.

[9] `https://pl.wikipedia.org/wiki/Problem_ośmiu_hetmanów`
[10] `https://en.wikipedia.org/wiki/Fork_bomb`